John The Ripper

Akintunde Ridwan Oladimeji
Student ID : 2025/INT/11578

Course Name : INT302 Kali Linux Tools and System Security

Instructor Name : Mr Aminu Idris (AMCPN)

Date of Submission : 7/1/2026
Version : 1.0

Table of Contents

1.0 Introduction
2.0 Aim and objectives
2.1 Tools
3.0 Methodology
3.1 Preparing the tools and files
3.2 Cracking password using default wordlists
3.3 Cracking password using custom wordlists
3.4 Cracking password using brute force mode
3.5 Cracking windows password hashes
3.6 Cracking passwords using Hashcat
4.0 Results and Analysis
5.0 Conclusion and Recommendation

Executive Summary

This lab focused on conducting password cracking using john the ripper and hashcat while studying their operational differences. Key activities include performing password cracking exercises using different attack techniques including dictionary and brute force attacks, comprehending various password hash formats and modes supported by john the ripper. The lab revealed that the hashcat cracked hashes faster than the john the ripper tool especially when loaded with the custom wordlist. Similarly the custom wordlist used in john the ripper happens to fast track the discovery process of the hash cracking among all the methods but has least successful rate compared to others when database of wordlist combinations are considered

1.0 Introduction

John the Ripper is a widely used password security auditing tool that analyzes password hashes by systematically attempting candidate passwords through wordlist, rule-based, mask, and incremental attacks, enabling security practitioners to evaluate password policy effectiveness and identify credentials vulnerable to compromise..

2.0 Aim and objectives

The aim of this lab is to conduct password hash cracking using various methods with john the ripper tool.

The Objectives of this lab is to

- Perform password cracking exercises using different attack techniques including dictionary and brute force attacks
- Analyze the success of these techniques and understand password complexity
- Understand how to use john the ripper to crack password hash
- Learn about various password hash formats and modes supported by john the ripper.

2.1 Tools

- John the Ripper
- Wordlist
- Hashcat

3.0 Methodology

3.1 Preparing the tools and files

- John the ripper application was installed using the command **sudo apt install john** and the installed version is **1.9.0-jumbo-1**
- The command **cat /etc/shadow** was run to check different types of hashed passwords in this file
- The command **john --list=format** was run to confirm the type of hash format handled by John the ripper
- Some txt files containing a different hash of were prepared to execute and study the john the ripper hash cracking operations using different techniques.

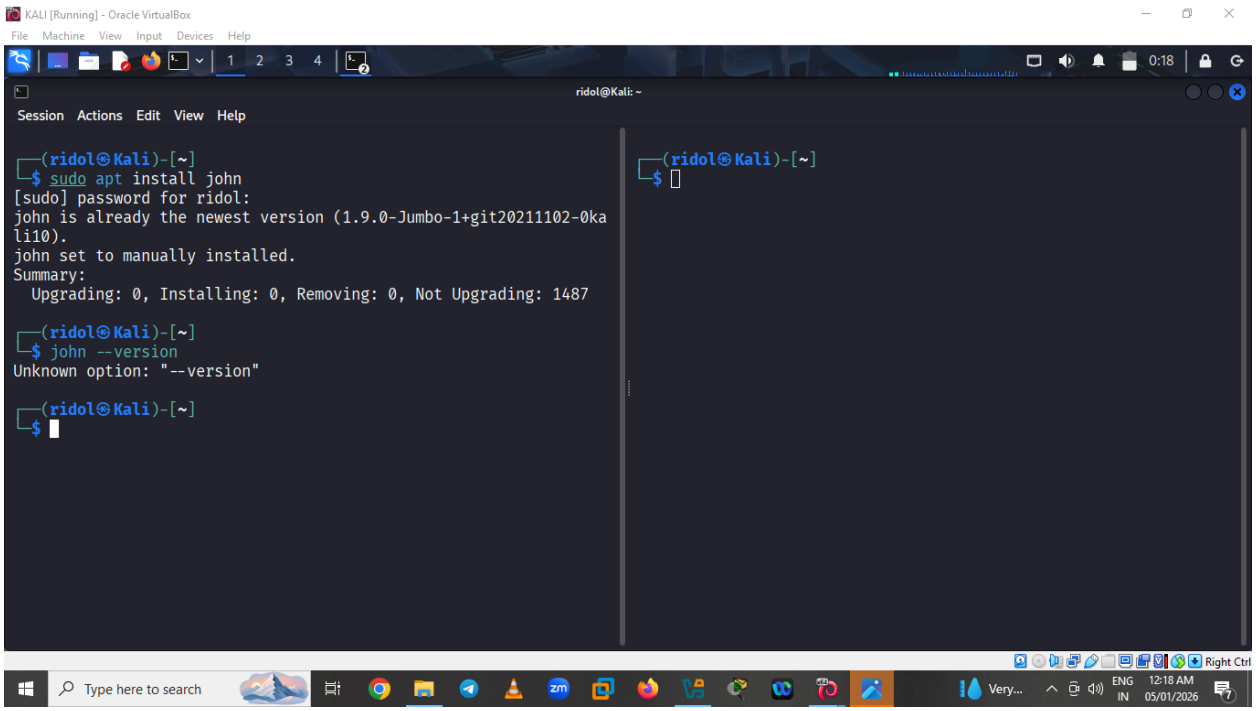

Fig 1: Evidence showing installation and version of john the ripper tool on the Linux terminal .

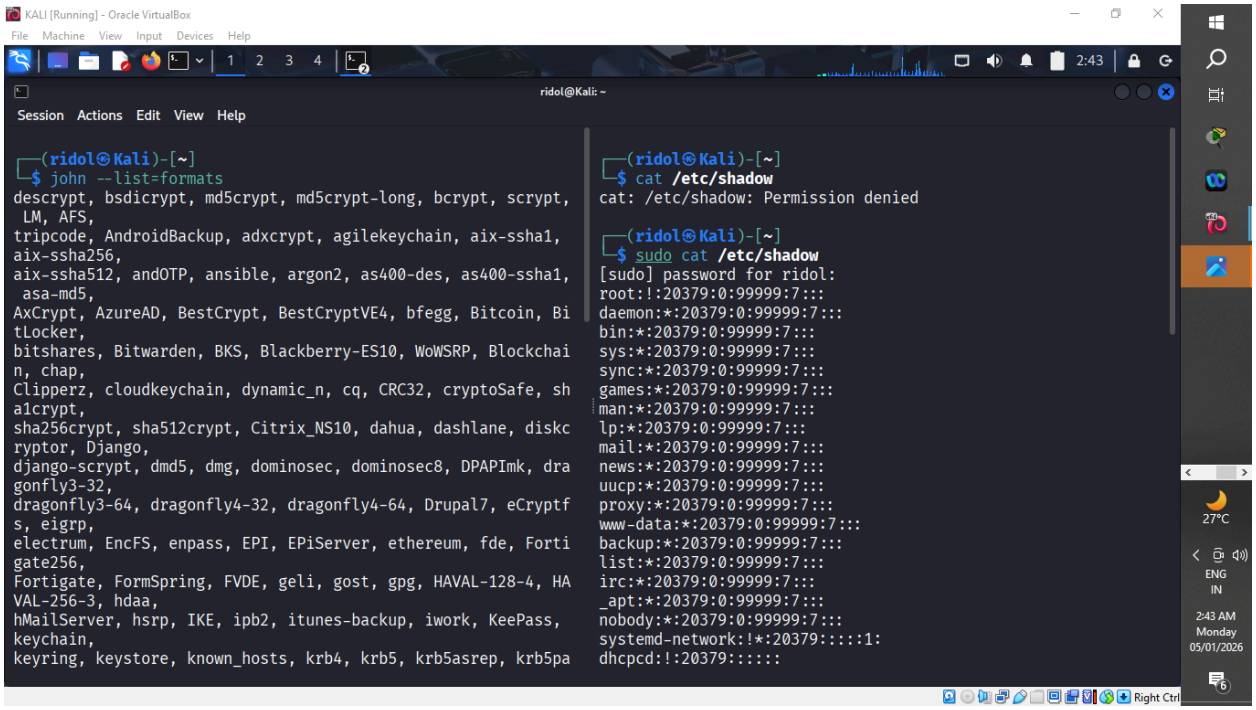

Fig 2: Evidence showing different types of hash format handled by John the ripper.

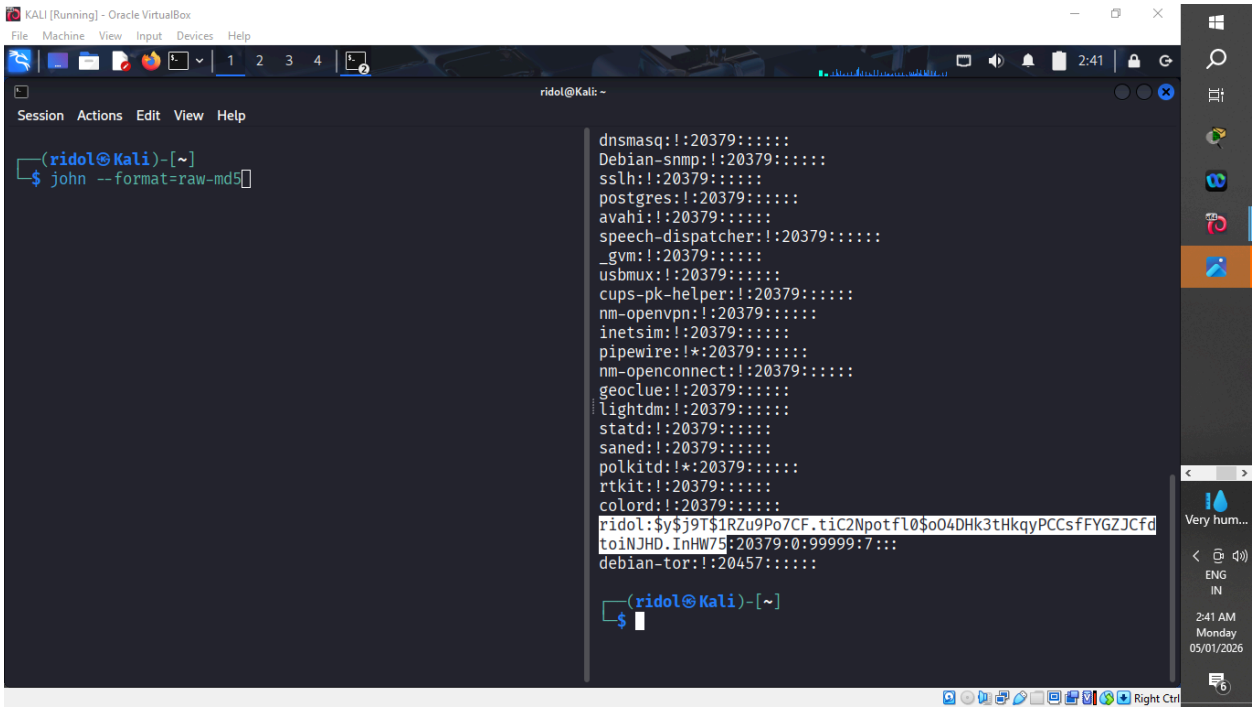

Fig 3: Evidence showing cat /etc/shadow output on Linux terminal

3.2 Cracking password using default wordlists

- The file of interest containing hash to be cracked was named **hash.txt**
- The command **john - -format =raw-md5 - -wordlist=/usr/share/wordlists/rockyou.txt hash.txt** was run using a pre-installed wordlist .
- The hash was cracked and the plaintext **password** was revealed as a match
- The step was repeated for several other hash file and all were successfully cracked to a plain text

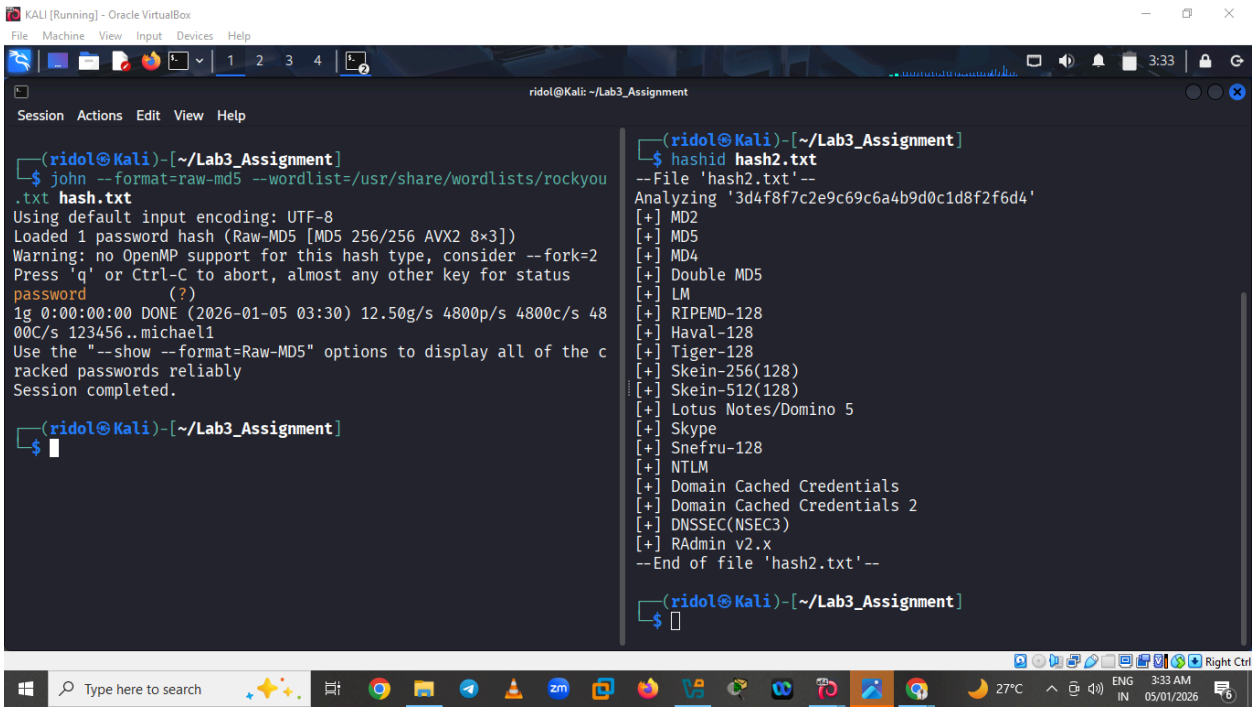

Fig 4: Evidence showing hash cracking by john the ripper tool on the Linux terminal .

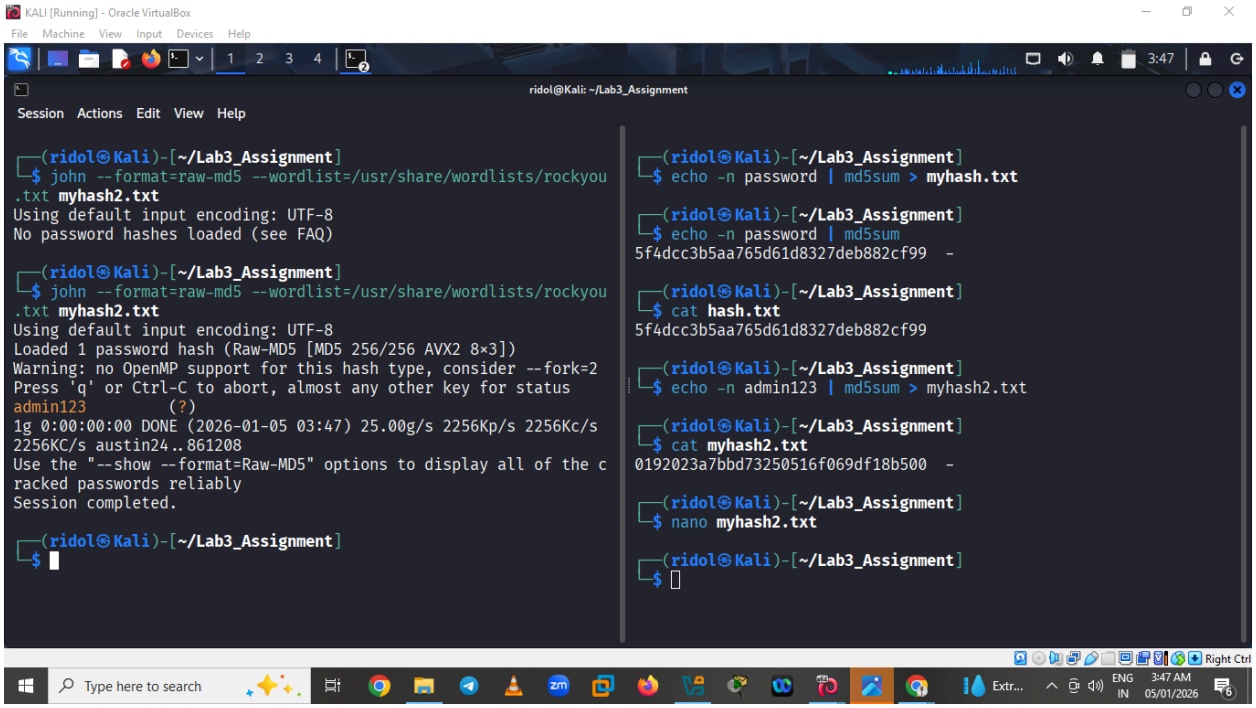

Fig 5: Evidence showing hash cracking by john the ripper tool on the Linux terminal .

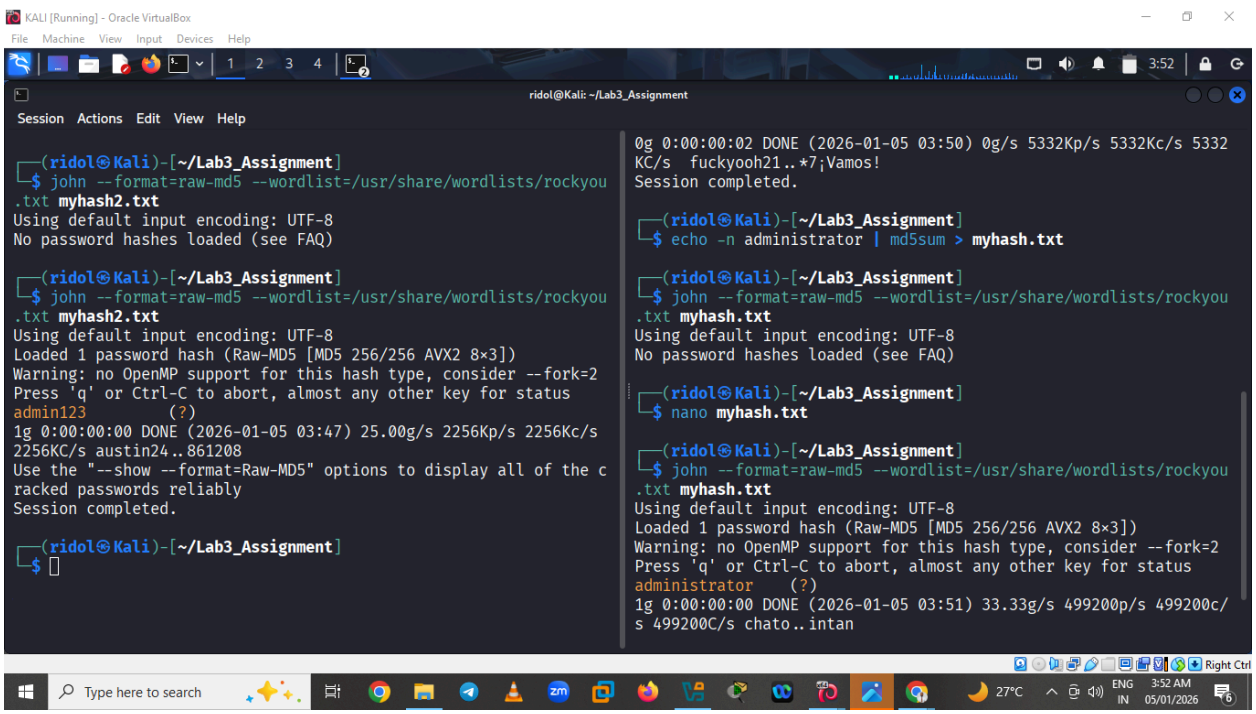

Fig 6: Evidence showing hash cracking by john the ripper tool on the Linux terminal .

3.3 Cracking password using custom wordlists

- A custom wordlist was prepared using the command **echo -e "password1\nletmein\nadmin123" > custom_wordlist.txt**
- The plaintext **Administrator-Ridol** was added to the wordlist using nano text editor while the hash was also generated and saved to **custom_Hash.txt** file.
- The file of interest containing hash to be cracked was named **custom_Hash.txt**
- The command **john --format=raw-md5 custom_Hash.txt --wordlist=custom_wordlist.txt** was run to initiate the crack process.
- The hash was cracked and the plaintext **Administrator-Ridol** was revealed as a match
- The step was repeated for several other hash files and all were successfully cracked to a plain text.

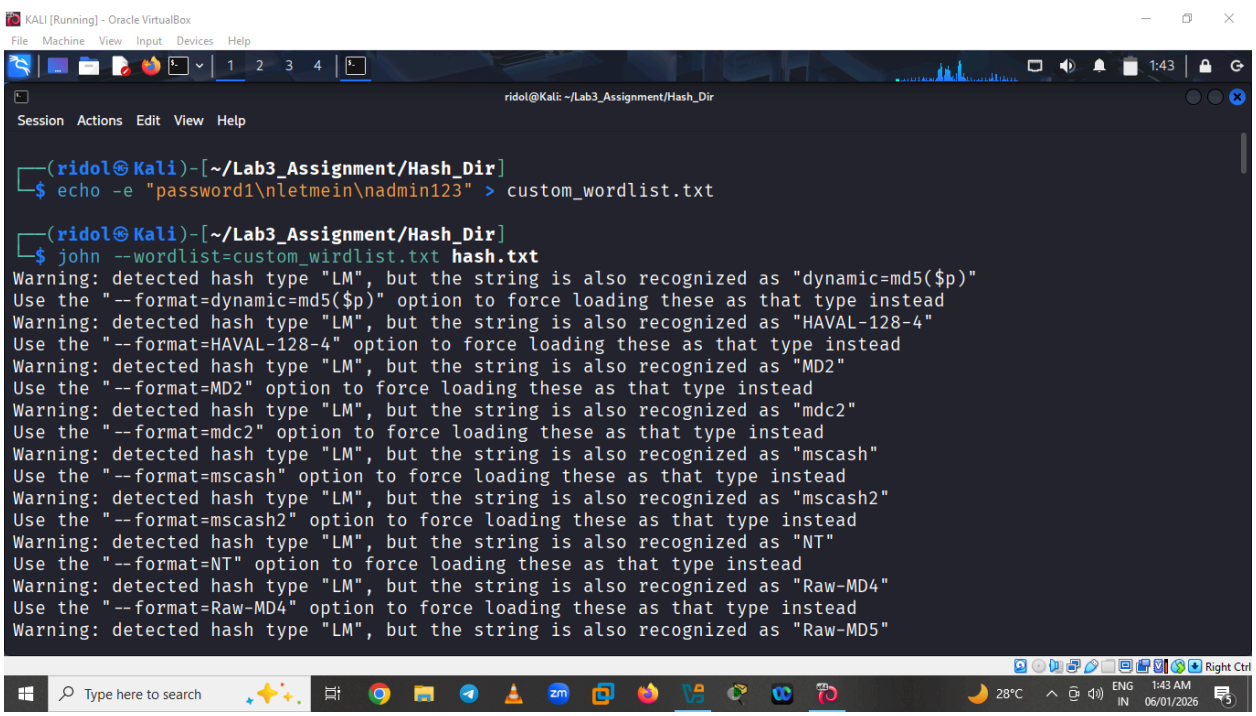

Fig 7: Evidence showing custom wordlist population on the Linux terminal .

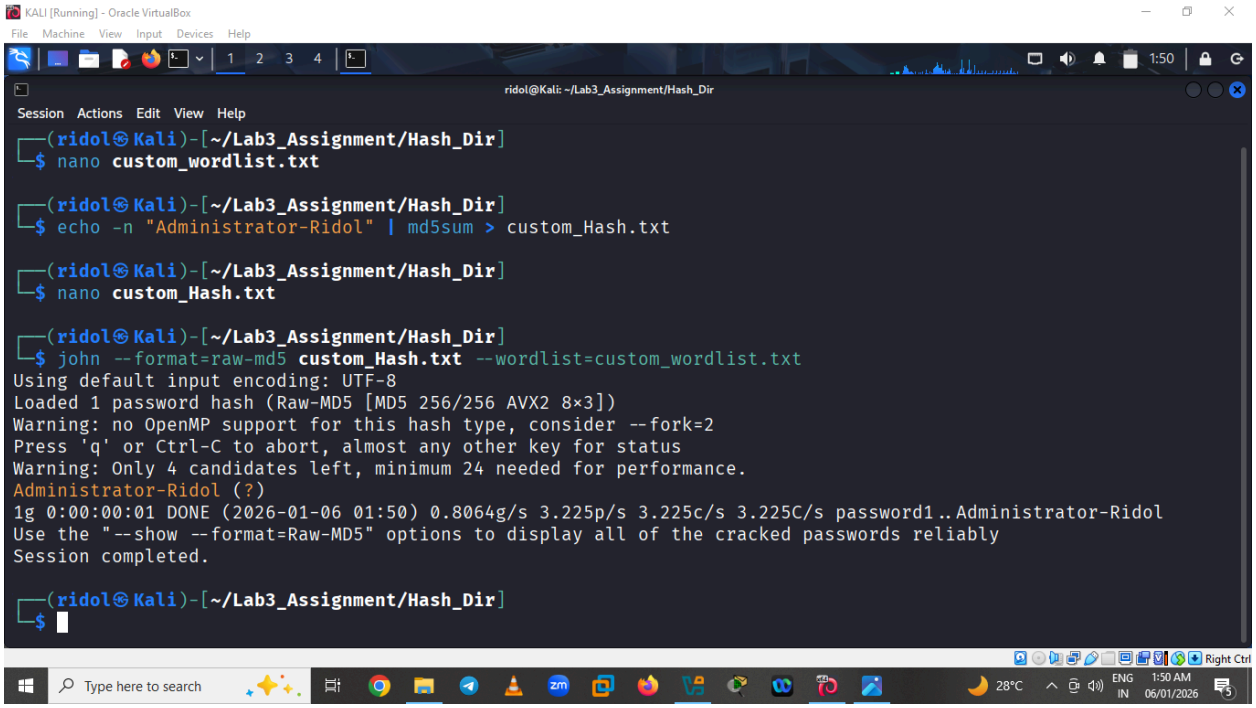

Fig 8: Evidence showing hash cracking using custom wordlist by john the ripper tool on the Linux terminal .

3.4 Cracking password using brute force mode

- The file of interest containing hash to be cracked was named **hash2.txt**
- The command **john --incremental --format=raw-md5 hash2.txt** was run and the brute force process commenced.
- The brute force process involved combining and testing several password combinations against the given hash to obtain a match and this took a long period of time.

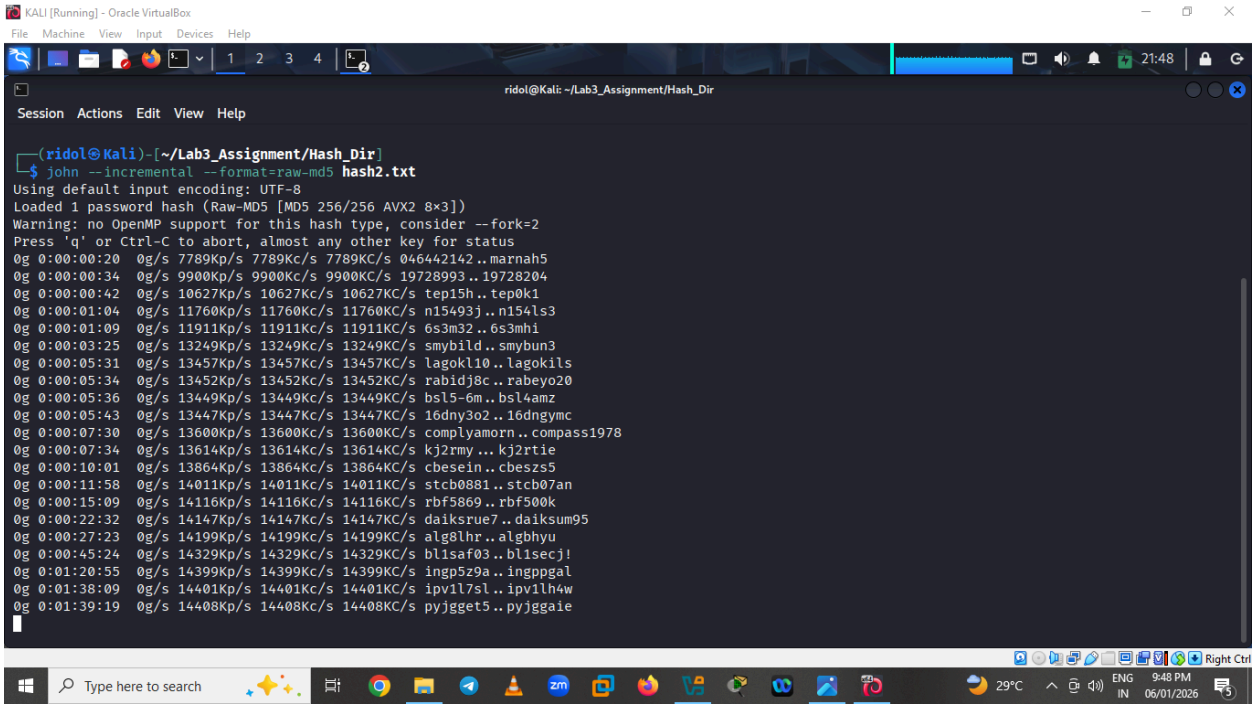

Fig 9: Evidence showing hash cracking using incremental mode by john the ripper tool on the Linux terminal .

3.5 Cracking windows password hashes

- The file of interest containing hash to be cracked was named **ntlm_hash.txt**
- The command **john - -format=nt ntlm_hash.txt - -wordlist=/usr/share/wordlists/rockyou.txt** was run to initiate the hash crack
- The command john - -test was run to check the possible operational duration of john the ripper.

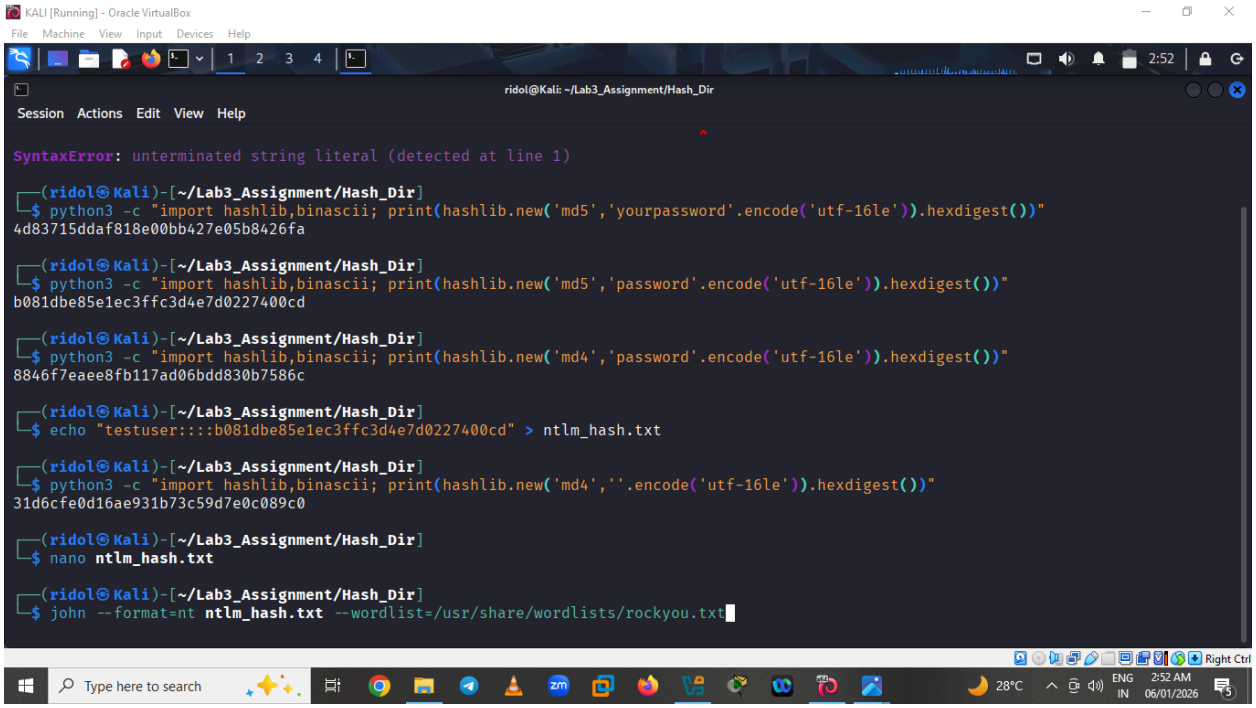

Fig 10: Evidence showing windows hash cracking by john the ripper tool on the Linux terminal .

3.6 Cracking passwords using Hashcat

- The hashcat was launched and the command **hashcat -h | grep md5** was run to confirm md5 attack mode and command types.
- The file of interest containing hash to be cracked was named **myhash2.txt** and the wordlist is **custom_wordlist.txt**
- The command **hashcat -m 0 -a 0 myhash2.txt custom_wordlist.txt -r /usr/share/hashcat/rules/best64.rule** was run to initiate the crack process
- The hash was cracked faster than john the ripper and the plaintext **admin123** was revealed as a match.
- The same process was repeated for another hash file and the plaintext was obtained much more quickly.

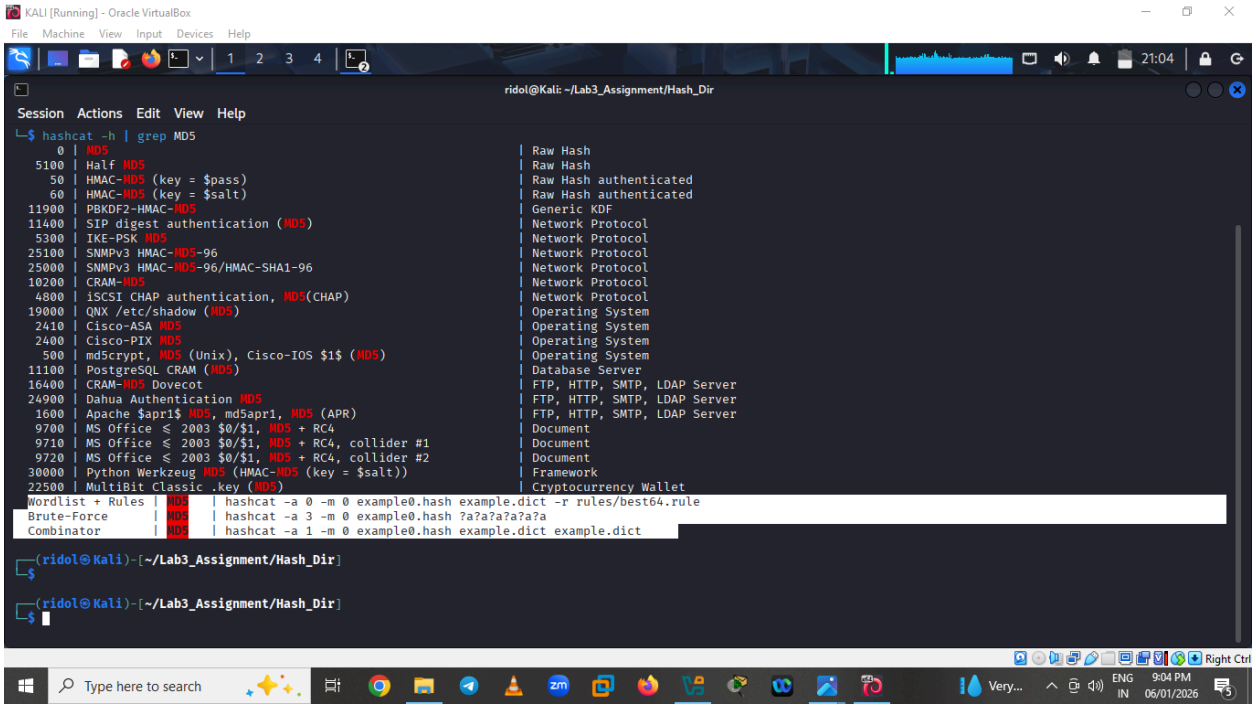

Fig 11: Evidence showing hashcat md5 formats on the Linux terminal .

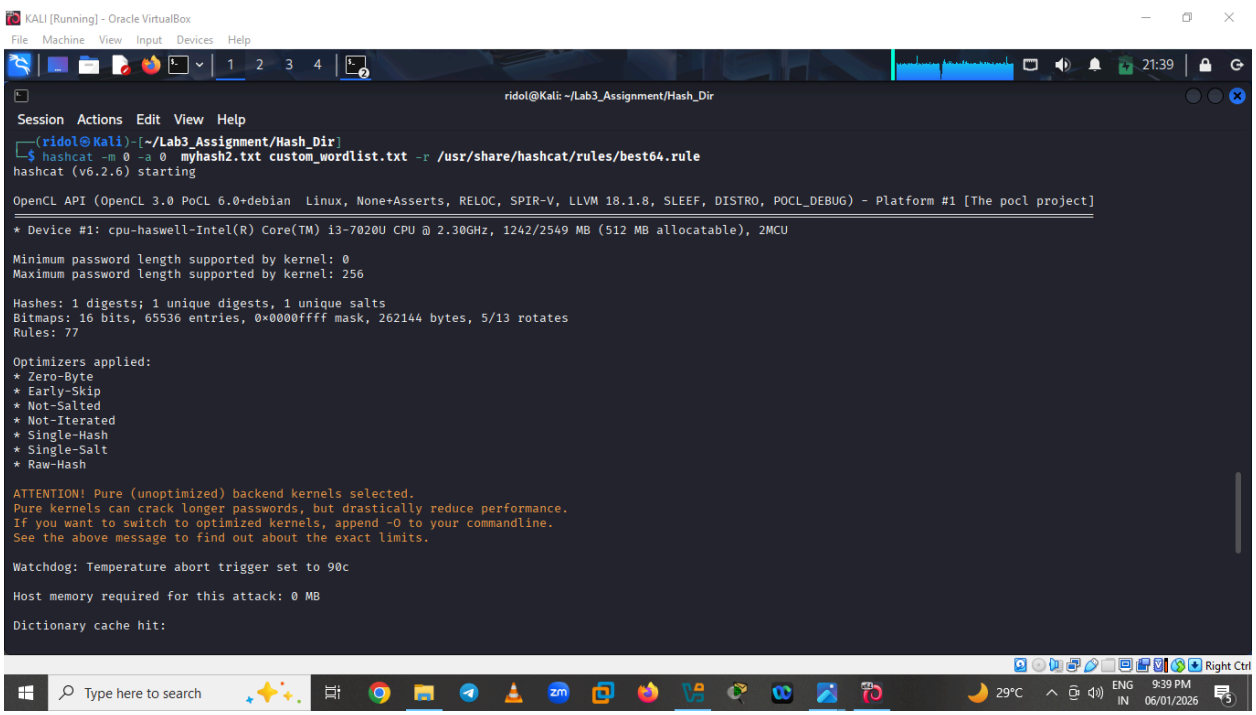

Fig 12: Evidence showing hash cracking by hashcat tool on the Linux terminal .

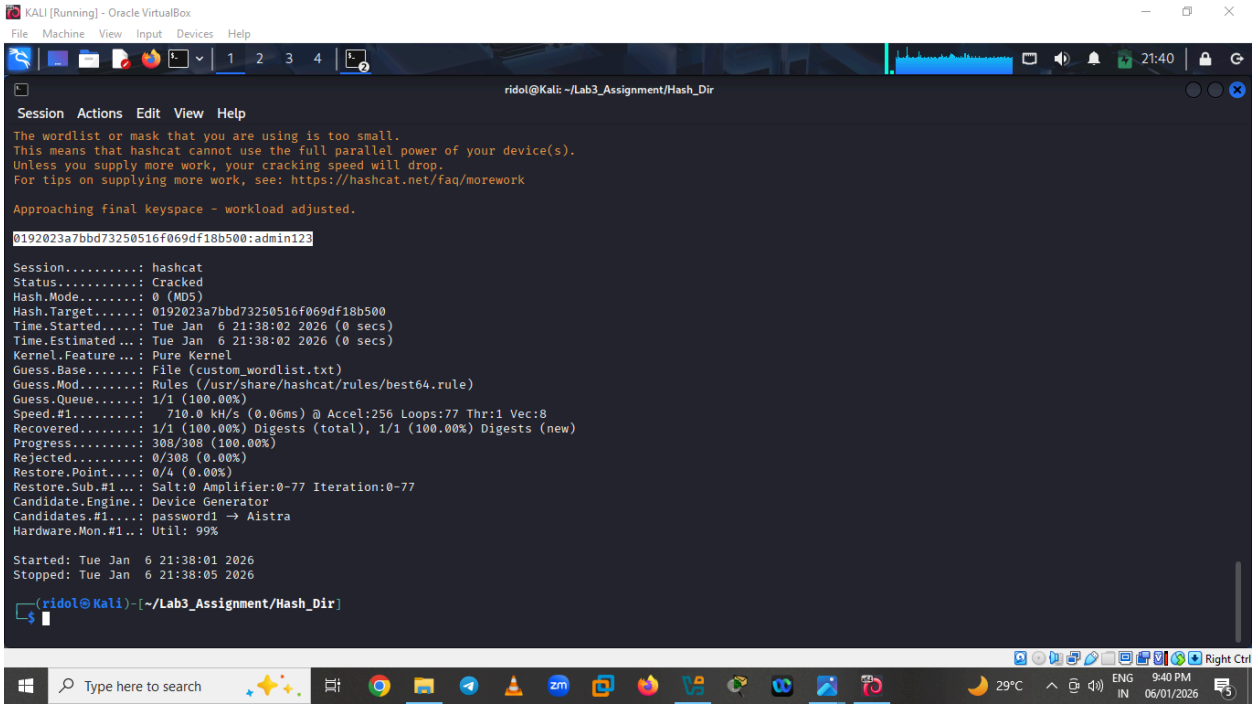

Fig 13: Evidence showing successful hash cracking by hashcat tool on the Linux terminal .

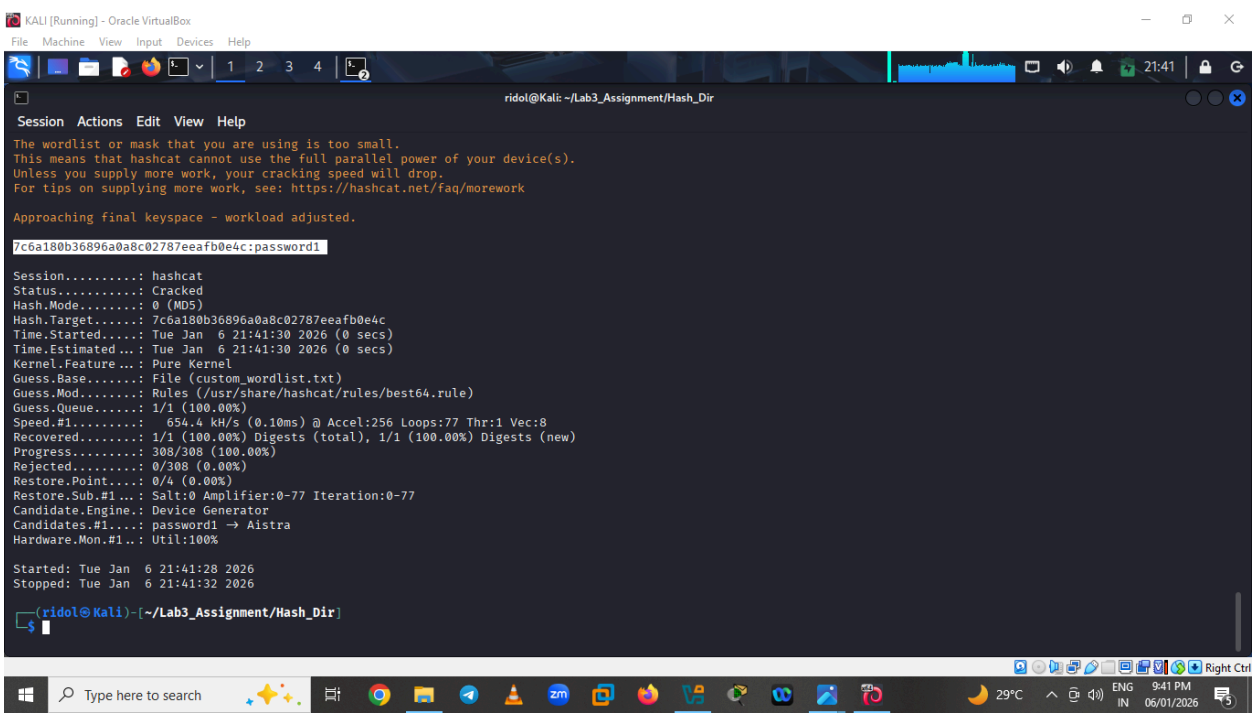

Fig 14: Evidence showing successful hash cracking by hashcat tool on the Linux terminal .

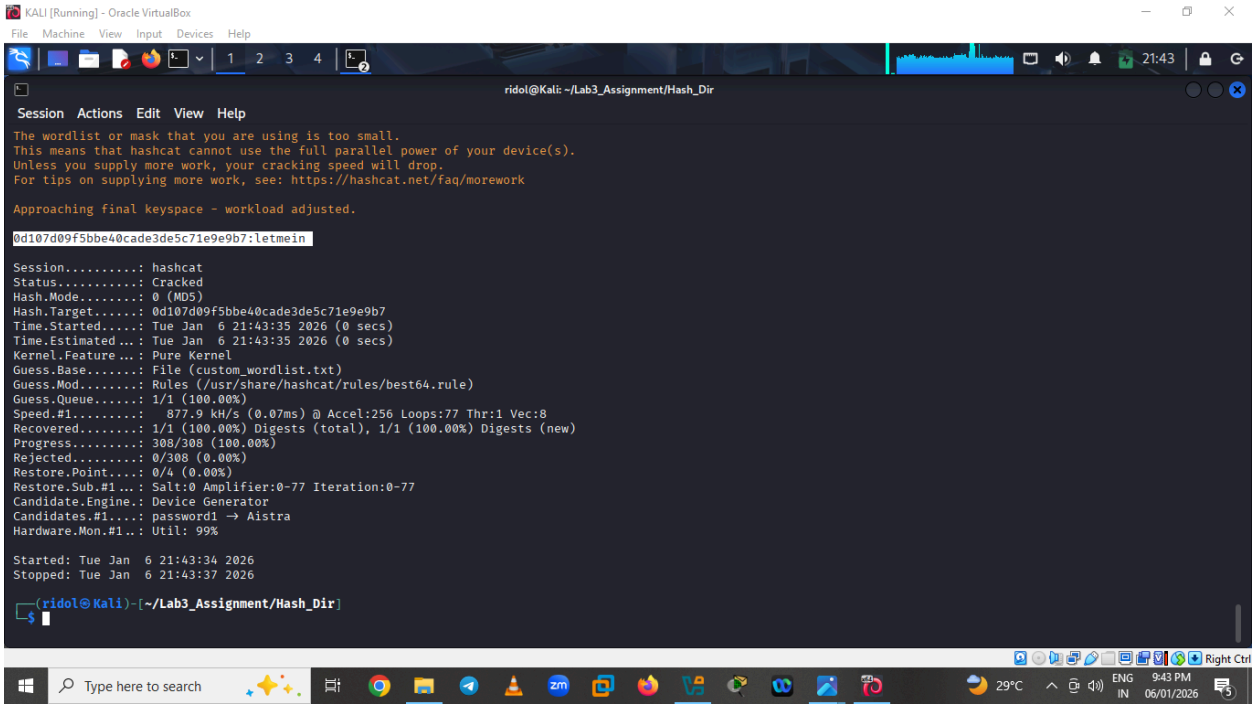

Fig 15: Evidence showing successful hash cracking by hashcat tool on the Linux terminal .

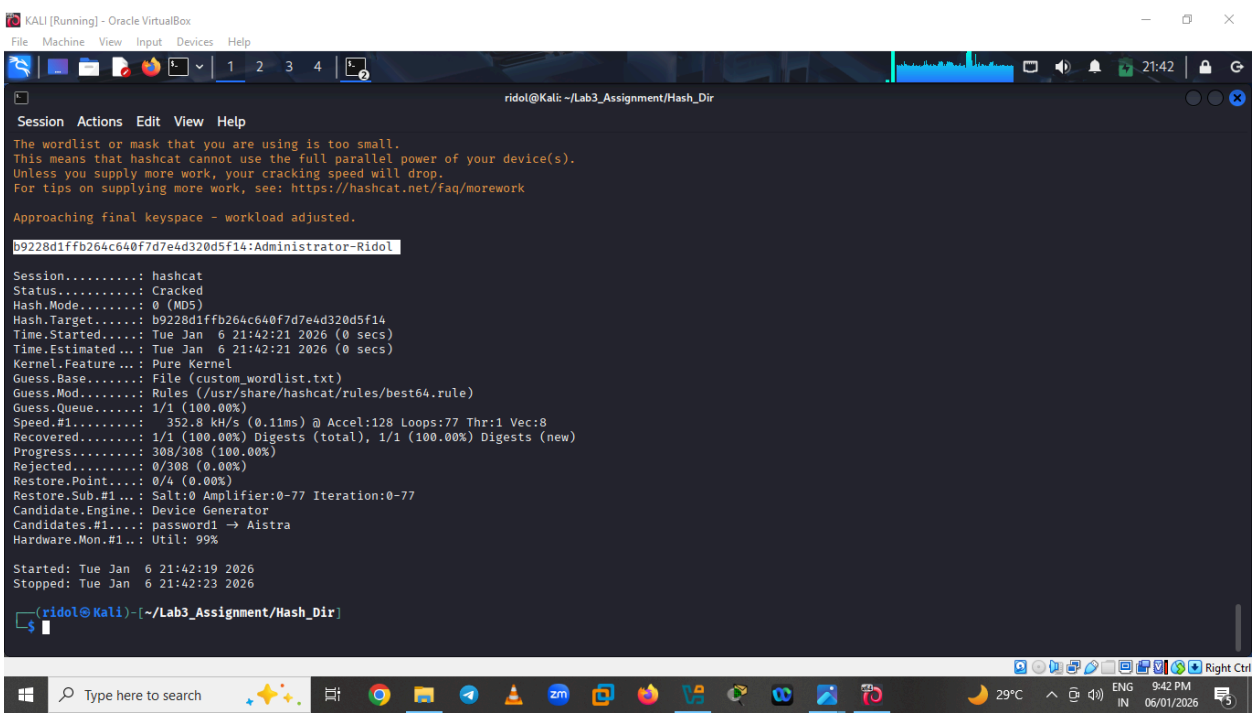

Fig 16: Evidence showing successful hash cracking by hashcat tool on the Linux terminal .

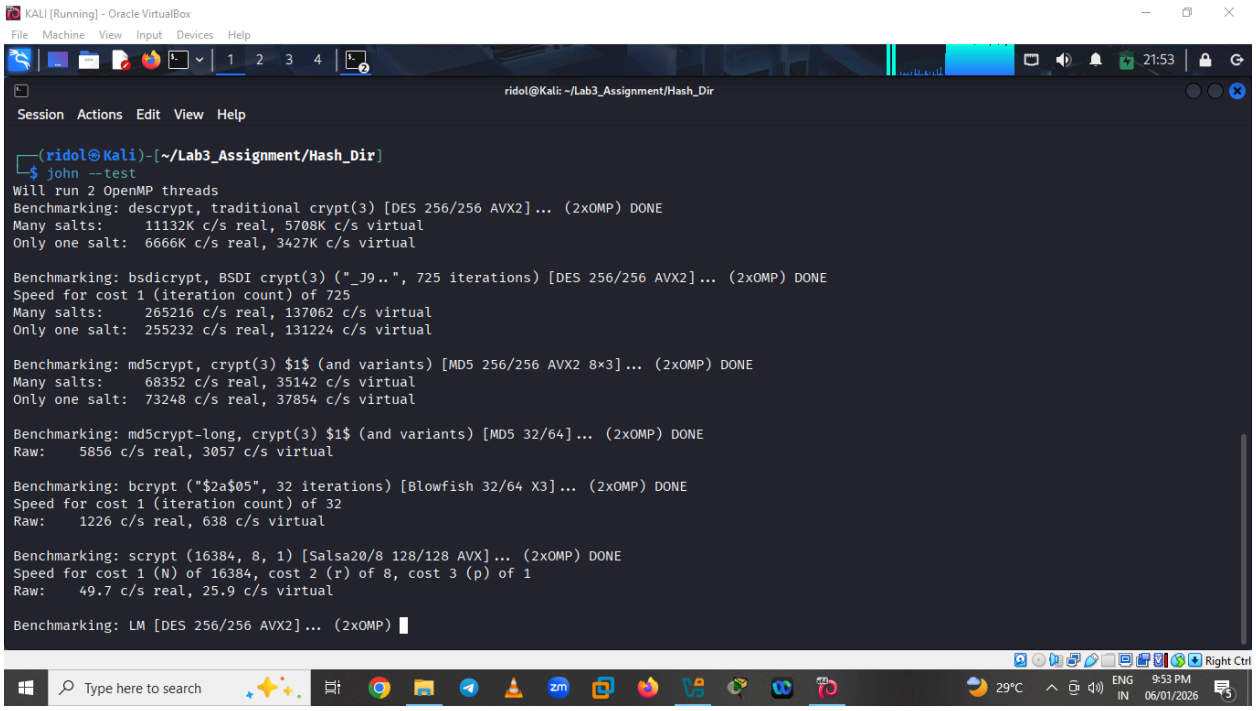

Fig 17: Evidence showing John the ripper speed test on the Linux terminal .

4.0 Results and Analysis

The hashcat cracked all the loaded hashes faster than the john the ripper tool especially when loaded with the custom wordlist. Similarly the custom wordlist loaded in john the ripper happens to fast track the discovery process of the hash cracking among all the methods.

The incremental mode tends to take a longer period of time in cracking the password hash. This can be attributed to the brute force method methods used in this mode. The default wordlist can be placed at the intermediate level between the custom wordlist and the brute force method.

It's important to note that the incremental format and default wordlist has a higher chance of success than a custom wordlist due to the wide area and large number of data of different possible combinations in their list than the custom wordlist although a lot of time will be required to achieve the cracking.

5.0 Conclusion and Recommendation

Password length, complexity and salt are important when choosing password combinations for any authentication process. An ideal password length should be between 8-12 characters minimum containing an uppercase letter, lower case letters , numbers and special characters. This would make it almost impossible for password cracking tools to crack or at the slightest possibility take an incredible amount of time long enough to discourage the attacker from cracking the password.